Software Implementation Procedure

Johan Smith Santamaria Fernández

Sharik Dayanna Rojas Ibarra

Juan Pablo Chala Ramírez

Technologist in Software Analysis and Development

Ficha: 3145556

SENA — National Learning Service

Neiva, Huila, Colombia, October 13, 2026

Software Implementation Procedure

Warning. This procedure describes the observed development environment, not a production deployment. Secrets must be configured in local `.env` files ignored by Git; they must never be copied into this document, screenshots, Git history, or error messages. The architecture, requirements, and functional status are detailed in Guardian Escolar Software Design; this document covers execution steps to avoid duplicating that description.

General information

The working repository is organized into separate folders for the frontend, gateway, microservices, database, and infrastructure. Each `docker-compose.yml` must be run from the folder containing it because it uses relative paths for the build context, `Dockerfile`, volumes, and `.env`.

No common commit/tag identifying an installable version of all components was found. For a formal delivery, record the hash or tag of each repository used and avoid mixing incompatible branches.

Prerequisites

Software

- Windows 10/11 with Docker Desktop and the Linux engine running; WSL2 is documented as the development environment.

- Docker Engine and Docker Compose V2 (`docker compose`).
- Git and access to the repositories.
- .NET SDK 10 to build/test ASP.NET Core services.
- JDK 21 for the Java IAM service.
- Node.js 20 and npm 10 for the web and mobile frontends; the web frontend image uses Node 20.
- An Android device or Expo-compatible emulator for verifying the mobile app. To test geolocation, grant explicit permissions and use test data.
- Internet connectivity to download images and packages during the initial installation, and to access SMTP/Twilio when validating those integrations.

Angular 21, Expo 57, React Native 0.86, SQL Server 2022, Kafka 4.1.0, Kong 3.9, and Liquibase 4.29 are versions/configurations observed in manifests or Compose files. Before updating a dependency, review compatibility in the component's repository.

Hardware

There is no evidence of a certified minimum specification or load test that would establish one. We recommend having sufficient memory and storage for Docker Desktop, SQL Server, Kafka, the gateway, and the Java/.NET projects, and reserving disk space for the database's persistent volume. Record the configuration used during the team's tests.

Obtaining the source code

The current working folder contains projects in separate directories. Clone each repository into an equivalent structure and keep folder paths and names consistent with the examples below. Do not assume that `GuardianEscolar` is the Git root for all services.

```
Proyecto\
├── database\
├── kafka\
├── sg-ms-api-gateway\
├── sg-ms-iam\
├── sg-ms-user-management\
├── sg-ms-school-management\
├── sg-ms-fleet\
├── sg-ms-route\
├── sg-ms-notification\
├── sg-ms-forgot-information\
└── GuardianEscolar\
    └── dvlp-front\
        ├── dvlp-web\
        └── dvlp-movil\guardian-escolar
```

Use the branch/tag agreed upon by the team in each repository. Before deployment, compare the gateway revision with each service's paths and verify that they use the expected API contracts.

The absolute paths in this procedure's commands correspond to the workstation where the document was prepared. On another machine, replace them with the local location of `Proyecto`.

Environment configuration

`.env` files

1. In each component that provides an `.env.example`, copy it to an `.env` file in that same folder. Templates are available for IAM, user management, schools, fleet, routes, notifications, recovery, and Expo. This checkout does not contain an `.env.example` for the database: create the local file using the variables described in step 4.

   From the component directory, create the file without overwriting an existing configuration:

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
```

2. Fill in the required values locally using development data. Do not put real secrets in an `.env.example`.
3. Check variable names against the component's Compose file and README. Compose interpolates `.env` from the Compose project directory.
4. For the database, configure `MSSQL_DATABASE`, `MSSQL_USER`, and `MSSQL_SA_PASSWORD`, as well as `SCHOOL_GUARDIAN_DB_DIR` only if the changelogs are not in the default path. The user defined in `MSSQL_USER` must be an existing SQL Server principal with migration permissions. Compose creates the database using `sa`, but Liquibase attempts to authenticate with `MSSQL_USER`; do not assume that a user named `user` is created automatically.
5. For IAM, user management, schools, fleet, routes, and notifications, configure the shared SQL Server connection and the messaging/authentication values required by their

`.env.example` files.

6. For recovery, provide the `ConnectionStrings__DefaultConnection` connection string, `INTERNAL_API_KEY`, `OTP_HASH_PEPPER`, and SMTP variables; add Twilio credentials only if phone-number changes will be tested.
7. For the web frontend, review `API_URL` and `API_FORGOT_INFORMATION_URL`.
8. For Expo, review `EXPO_PUBLIC_API_URL` and `EXPO_PUBLIC_FORGOT_INFORMATION_API_URL`. These URLs are accessible in the client bundle; do not use them for secrets.

Do not publish connection values, passwords, JWT secrets, API keys, OTP pepper, SMTP passwords, or Twilio credentials. On a physical device, `localhost` points to the phone itself: the API must use an address accessible from the mobile device, such as the computer's private IP on the same network or an active HTTPS tunnel. The private IP may change when reconnecting to the network.

Database setup

The inspected configuration uses **Microsoft SQL Server 2022**, not PostgreSQL. Schemas and migrations are under database; Liquibase applies the domain changelogs.

1. Verify that the `.env` file exists in `database` and contains the required values, without printing them in shared terminals.
2. Ensure that the external Docker network exists. If it has not been created yet:

```
docker network inspect sg-services-network
# If the command confirms that it does not exist:
docker network create sg-services-network
```

3. Start SQL Server and the initialization/migration services from the `database` folder:

```
Set-Location &lt;RUTA_LOCAL&gt;
docker compose up -d
```

4. First check the SQL Server container status, then check the `liquibase-*` logs. Do not start the APIs if any required migration failed.

```
docker ps -a
docker logs --tail 100 school-guardian-db-dev
docker logs --tail 100 liquibase-iam
```

5. Keep the `school-guardian-sqlserver-data` volume if the local database must be preserved. Do not routinely delete volumes as a troubleshooting method; doing so may erase data.

The database Compose file defines dependencies between migrations. If a changeset fails, first review the specific error and the Liquibase history table. Do not edit a changeset that has already been applied in a shared environment without a reviewed corrective migration.

Backend installation and configuration

Gateway and services

Services communicate by container name over `sg-services-network`; internal traffic typically uses port 8080. The gateway's HTTP entry point is published on port 8000. For each service, open PowerShell and run the following from the corresponding directory:

```
Set-Location &lt;RUTA_LOCAL&gt;
```

```
docker compose up -d --build
```

Repeat from `sg-ms-user-management`, `sg-ms-school-management`, `sg-ms-fleet`, `sg-ms-route`, and `sg-ms-notification`, only after confirming that SQL Server and its migrations are ready. Start `sg-ms-forgot-information` after IAM because it uses IAM's internal endpoints.

```
Set-Location &lt;RUTA_LOCAL&gt;
docker compose up -d --build
```

Start Kafka from its own repository if the component's workflow requires events:

```
Set-Location &lt;RUTA_LOCAL&gt;
docker compose up -d
```

Start the gateway from `sg-ms-api-gateway` after the required upstreams have DNS names on the network:

```
Set-Location &lt;RUTA_LOCAL&gt;
docker compose up -d
```

If a service has no `.env.example` or its Compose file does not define a fixed host port, do not invent values: consult that service's README and Compose file. Several Compose files expose `8080` as an internal port without a fixed host mapping; APIs are accessed through Kong or internal DNS, not through a random port.

Password recovery

IAM and recovery must point to the same development SQL database. The local `INTERNAL_API_KEY` value must be identical in both services; IAM exposes its `/api/profiles/**` routes only as internal endpoints. Testing email delivery requires an

SMTP server and an authorized account. Changing a phone number requires Twilio Verify to be configured. Without those dependencies, limit verification to tests that do not send messages or modify real data.

Startup logs and observability

Use these commands without displaying environment variables:

```
docker compose ps
docker logs --tail 100 ms-iam
docker logs --tail 100 ms-user-management
docker logs --tail 100 ms-school-management
docker logs --tail 100 ms-fleet
docker logs --tail 100 ms-route
docker logs --tail 100 ms-notification
docker logs --tail 100 ms-forgot-information
docker logs --tail 100 kong
```

Specific container names may change depending on the Compose version. If `docker logs` reports that a container does not exist, check its name with `docker ps -a`.

Web frontend installation

The frontend is located in `GuardianEscolar/dvlp-front/dvlp-web`. Its `package.json` declares npm, Angular 21, and the `start`, `build`, and `test` scripts.

```
Set-Location &lt;RUTA_LOCAL&gt;
npm ci
npm start
```

Set `API_URL` to the gateway's base URL and `API_FORGOT_INFORMATION_URL` to the recovery service URL if it is exposed directly. To build and run tests:

```
npm run build
```

```
npm test -- --watch=false
```

The `GuardianEscolar` Compose file publishes the development web app on port 4200. The production Dockerfile builds the static Angular app and serves it with Nginx; to create an image for a specific environment, configure the API before building and check which values are included in the bundle.

Mobile app installation

The mobile project is at GuardianEscolar/dvlp-front/dvlp-movil/guardian-escolar.

```
Set-Location &lt;RUTA_LOCAL&gt;
npm ci
npm start
```

Expo loads `EXPO_PUBLIC_*` variables when preparing the bundle; reload the app after changing the URLs. During development on the same Wi-Fi network, set both URLs to a host/IP address the phone can reach and ensure the Windows firewall allows the required ports. Do not use `localhost` on a physical device.

The project provides `start:lan`, `android`, `ios`, and `web` scripts. On Windows, iOS requires Apple tools that are not natively available. The `sync-api-tunnels.mjs` script can synchronize and test Cloudflare URLs when the `.env` already contains tunnel URLs; Quick Tunnels generate temporary URLs and are not a stable endpoint.

Startup order and ports

Recommended startup order

1. Docker Desktop and the `sg-services-network` network.

2. SQL Server and Liquibase migrations.

3. Kafka, when required by the services being validated.

4. IAM and the backend services required for the test.

5. Recovery service, after IAM, if that flow will be tested.

6. Kong API Gateway.

7. Web frontend, Expo app, and finally device testing.

Container availability does not guarantee that its dependencies are available; check logs and endpoints before each integration step.

Known ports

Host port	Component	Use/notes
4200	Web development frontend	Published by the `GuardianEscolar` Compose file.
8000	Kong HTTP proxy	Gateway base URL in development.
8001, 8002	Kong Admin API/UI	Administrative ports published by Compose; keep them development-only and do not expose them to untrusted networks.
8080	Backend APIs inside the Docker network	Several Compose files do not define a host port for each API.
8091	Recovery service	Published as `8091:8080` in its Compose file.
9092	Kafka	Published by the broker's Compose file; its listener is configured for the Docker network.

Host port	Component	Use/notes
1433	SQL Server	Published locally for administration/testing; protect it with a firewall and development credentials.
9081	Metro/Expo, depending on Compose	Port configured in the mobile development Compose file; check the startup mode in use.

Implementation verification

Infrastructure checks

```
docker network inspect sg-services-network
docker ps -a
Invoke-RestMethod http://localhost:8091/health
```

The `/health` endpoint is defined by the recovery service. For other services, consult their README/OpenAPI documentation and do not assume they all expose the same health-check path.

API and application checks

1. Open the web frontend URL and confirm that it loads.
2. Check that `API_URL` in the web app and `EXPO_PUBLIC_API_URL` in the mobile app point to Kong and are accessible from the browser or phone.
3. Send a sign-in request only with an authorized test account. IAM's endpoint is configured under `/api/v1/auth`; do not log passwords or tokens.
4. Validate permissions using accounts with different roles and test resources. Check responses from both the API and the interface.

5. Test CRUD operations only on development records and verify persistence in the corresponding schema.
6. For recovery, follow the safe tests in TESTING.md; steps that change a real password require explicit authorization.
7. For each test, record the date, commit/tag, commands, service, result, and sanitized evidence.

The presence of a screen does not replace a backend permission test. GPS display can be marked as approved only if an authorized provider or simulator, ingestion, persistence/lookup, and visible end-to-end updates have all been verified.

Common troubleshooting

Symptom	Initial diagnosis	Safe action
The mobile app reports that it has no connection	Empty/outdated URL, `localhost` on a physical device, expired Quick Tunnel, or phone outside the LAN.	Correct `EXPO_PUBLIC_API_URL`, test connectivity to the host IP/port, and reload Expo. If using a tunnel, update the URL with the synchronization script and verify the gateway.
HTTP 502 during login through Kong	Kong is running, but an upstream is unavailable or cannot connect to SQL Server.	Check IAM, SQL, and migration status/logs; verify that containers share `sg-services-network` and that the upstream name/port matches.
Recovery API does not respond	SQL Server or IAM has not started yet, a required `.env` value is missing, or OTP schema creation failed.	Check `docker compose ps`, logs, the connection string, and `/health`; do not publish `.env`.

Symptom	Initial diagnosis	Safe action
IAM does not start	Datasource error, missing JWT key, or failed Liquibase migration.	Confirm that SQL is healthy and check the connection variable. Review the failed changeset. Do not delete data or edit Liquibase history without a recovery plan.
`FK_Profile_Person` is duplicated during migration	Database state and Liquibase history do not match, or a changeset was applied manually/partially.	Stop destructive retries; compare the SQL object with `DATABASECHANGELOG` and apply a versioned correction reviewed by the data owner.
A container shuts down	The main process exited; Compose does not necessarily restart every service.	Inspect the container and its logs; distinguish `OOMKilled`, configuration failure, and unavailable dependency. Fix the cause before adding a restart policy.
Port is in use	Another process or container is using the requested host port.	Run `docker ps` and `Get-NetTCPConnection -LocalPort <port>`; stop only the identified process/service or coordinate changes to the published port and client URL.
Migrations do not start	External Docker network is missing, SQL is not healthy yet, or a prerequisite changeset failed.	Inspect the network, health check, and Liquibase logs; run from the correct folder and respect dependencies.
Emails or SMS messages are not delivered	SMTP/Twilio is not configured, credentials are incorrect, the provider rejects delivery, or the test account is not enabled.	Review local configuration and sanitized logs; confirm provider permissions. Do not repeatedly send messages to users without authorization.

Updates, maintenance, and shutdown

Update

1. Agree on a compatible branch and set of versions.
2. Back up SQL Server before schema changes; document the backup location and verification without including credentials.
3. Review and apply approved Liquibase changelogs.
4. Build and update affected services with `docker compose up -d --build` from each service folder.
5. Review logs and smoke tests, and update the documentation/matrix if functionality has changed.

Review logs

Use `docker logs --tail 100 <container>` and filter out any personal data before sharing. Do not attach `.env` files, tokens, OTPs, passwords, or complete connection values.

Stop the environment

Run `docker compose down` from each project whose container set should be stopped. To preserve SQL Server, use Compose from the appropriate folder and do not add `--volumes`. Do not use `docker compose down -v` as a routine action: it would delete persistent volumes.

Recovery

In case of failure, preserve logs and container state, validate health checks and

dependencies, and restore from a verified backup if data was affected. Do not rebuild or recreate a shared development database without first confirming that it does not contain data the team needs.

Appendices

Variables by component

The exact variable names and whether they are required are specified by each environment template and Compose file. The main categories are:

- SQL Server/Liquibase: user, database, local password, and changelog path.
- IAM and microservices: SQL connection, authentication secret, and Kafka/gRPC settings as required by each service.
- Recovery: connection string, shared internal key, OTP pepper, SMTP, and optional Twilio credentials.
- Web: gateway URL and recovery URL.
- Mobile: gateway URL and recovery URL, with no secrets.

Repositories and technical documentation

Services are separated into `sg-ms-*`, the database is in `database`, the gateway is in `sg-ms-api-gateway`, the broker is in `kafka`, and clients are in `GuardianEscolar/dvlp-front`. For the catalog of capabilities, roles, architecture, and data, see Guardian Escolar Software Design; for background and APA-style references, see

Collaborative article.